10.2. Circular Queues

Previous Sections:

10. Queues

10.1. Operations on Queues

Contents:

In a linear queue, elements are inserted at the rear and removed from the front. While this follows the FIFO principle perfectly, it introduces an inefficiency when implemented using a fixed-size array.

Consider the following situation:

```
Initial Queue

FRONT                          REAR
```

```
  ↓                             ↓
+------+------+------+------+------+
|  A   |  B   |  C   |  D   |  E   |
+------+------+------+------+------+
```

After several dequeue operations:

```
FRONT                 REAR
  ↓                     ↓
+------+------+------+------+------+
|      |      |  C   |  D   |  E   |
+------+------+------+------+------+
```

Notice that the memory locations previously occupied by A and B are now empty. However, in a traditional linear queue, we continue inserting only after REAR.

Eventually, REAR reaches the end of the array:

```
FRONT                         REAR
  ↓                            ↓
+------+------+------+------+------+
|      |      |  C   |  D   |  E   |
+------+------+------+------+------+
```

At this point, even though there are empty spaces at the beginning, no new element can be inserted. The queue appears full, resulting in wasted memory.

To solve this problem, we introduce the **Circular Queue**.

Instead of treating the queue as a straight line, we connect the last position back to the first position, forming a circle. When REAR reaches the end of the array, it can wrap around and continue inserting into available spaces at the beginning.

A circular queue can be visualized as:

```
                  FRONT
                    ↓

              +---------+
            /             \
          [A]               [H]
         /                    \
       [B]                    [G]
       |                        |
```

```
        |                           |
        [C]                       [F]
          \                       /
           [D]                 [E]
              \               /
                +---------+

                     ↑
                   REAR
```

This simple idea allows us to reuse freed memory locations efficiently without shifting elements.

1. Characteristics of a Circular Queue

A circular queue still uses two pointers: FRONT and REAR. However, when either pointer reaches the end of the array, it wraps around to the beginning.

For example:

```
Index:

  0    1    2    3    4    5    6    7
+----+----+----+----+----+----+----+----+
| A  | B  | C  | D  | E  | F  | G  | H  |
+----+----+----+----+----+----+----+----+
                            ↑          ↑
                         FRONT       REAR


                +---------+
             /               \
           [A]                 [H] ← REAR
         /                        \
        [B]                       [G]
        |                           |
        |                           |
        [C]                       [F] ← FRONT
          \                       /
           [D]                 [E]
              \               /
                +---------+
```

The next insertion does not move beyond index 7. Instead:

```
Index:

  0    1    2    3    4    5    6    7
+----+----+----+----+----+----+----+----+
| A  | B  | C  | D  | E  | F  | G  | H  |
+----+----+----+----+----+----+----+----+
  ↑                        ↑
 REAR                    FRONT


                +---------+
             /              \
  REAR →   [A]                [H]
         /                      \
       [B]                      [G]
       |                          |
       |                          |
       [C]                      [F] ← FRONT
         \                      /
           [D]                [E]
              \              /
                +---------+
```

The rear wraps around to the beginning.

2. Overflow Conditions

A circular queue is considered full when no more free locations exist. This occurs in either of the following situations:

- Case 1: FRONT is at the beginning and REAR is at the last index.

```
  0    1    2    3    4    5    6    7
+----+----+----+----+----+----+----+----+
| A  | B  | C  | D  | E  | F  | G  | H  |
+----+----+----+----+----+----+----+----+
                          ↑              ↑
                        FRONT          REAR
```

- Case 2: REAR is immediately before FRONT after rotation.

```
  0    1    2    3    4    5    6    7
+----+----+----+----+----+----+----+----+
| A  | B  | C  | D  |    | F  | G  | H  |
+----+----+----+----+----+----+----+----+
```

```
                ↑         ↑
                REAR      FRONT
```

Mathematically: `FRONT = REAR + 1`. When either condition occurs, the queue is full and an overflow occurs.

3. Underflow Conditions

A circular queue is empty when: `FRONT = REAR = -1`

Another special case occurs when only one element remains:

```
FRONT
  ↓
+----+
| A  |
+----+
  ↑
REAR
```

After removing that element: `FRONT = REAR = -1` . The queue returns to its initial empty state.

4. Enqueue in a Circular Queue

When inserting an element:

1. Check for overflow.
2. If the queue is empty, assign FRONT = REAR = 0.
3. If REAR reaches the last index, wrap it back to index 0.
4. Otherwise, increment REAR normally.
5. Insert the element.

Example:

```
Before Insert

  0    1    2    3    4    5    6    7
+----+----+----+----+----+----+----+----+
| A  | B  | C  | D  | E  | F  |    |    |
+----+----+----+----+----+----+----+----+
```

```
  ↑                       ↑
FRONT                   REAR
```

```
Insert G:

  0    1    2    3    4    5    6    7
+----+----+----+----+----+----+----+----+
| A  | B  | C  | D  | E  | F  | G  |    |
+----+----+----+----+----+----+----+----+
  ↑                            ↑
FRONT                        REAR
```

5. Dequeue in a Circular Queue

When removing an element:

1. Check whether the queue is empty.
2. Retrieve the element at FRONT.
3. Move FRONT to the next position.
4. If FRONT reaches the last index, wrap it back to index 0.
5. If the removed element was the last element, reset FRONT and REAR to -1.

Example:

```
Before Delete

  0    1    2    3    4    5    6    7
+----+----+----+----+----+----+----+----+
| A  | B  | C  | D  | E  | F  | G  | H  |
+----+----+----+----+----+----+----+----+
                           ↑         ↑
                         REAR      FRONT
```

```
Remove A:

  0    1    2    3    4    5    6    7
+----+----+----+----+----+----+----+----+
| A  | B  | C  | D  | E  | F  | G  |    |
+----+----+----+----+----+----+----+----+
```

```
  ↑                          ↑
FRONT                      REAR
```

Shifting is required since we reach the last index.

6. Traversing a Circular Queue

Traversal begins at FRONT and continues until REAR is reached. The important difference from a linear queue is that traversal must wrap around when it reaches the end of the array.

Example:

```
  0    1    2    3    4    5    6    7
+----+----+----+----+----+----+----+----+
| A  | B  | C  | D  |    |    | E  | F  |
+----+----+----+----+----+----+----+----+
                 ↑                   ↑
               REAR                FRONT
```

Traversal order: ***E*** → ***F*** → ***A*** → ***B*** → ***C*** → ***D***

Notice how the traversal wraps from index 5 back to index 0.

7. Python Implementation of a Circular Queue

```python
class CircularQueue:
    def __init__(self, maximum=100):
        self.maximum = maximum
        self.front = -1
        self.rear = -1
        self.items = [None] * maximum

    def is_empty(self):
        return self.front == -1

    def remove(self):
        if self.front == -1:
            return False
```

```
        if self.front == self.rear:
            self.front = -1
            self.rear = -1

        else:
            if self.front == self.maximum - 1:
                self.front = 0
            else:
                self.front += 1

        return True

    def enqueue(self, data):
        if ((self.front == 0 and self.rear == self.maximum
- 1)
                or (self.front == self.rear + 1)):
            print("Queue Overflow")
            return False

        if self.front == -1:
            self.front = 0
            self.rear = 0

        else:
            if self.rear == self.maximum - 1:
                self.rear = 0
            else:
                self.rear += 1

        self.items[self.rear] = data
        return True

    def dequeue(self):
        if self.front != -1:
            temp = self.items[self.front]
            self.remove()
            return temp
```

```
            print("Queue is empty")
            return False

        def traverse(self):
            front_index = self.front
            rear_index = self.rear

            if front_index == -1:
                print("Queue is empty")
                return

            if front_index <= rear_index:
                while front_index <= rear_index:
                    print(f"{self.items[front_index]} <-", end
=" ")
                    front_index += 1

            else:
                while front_index <= self.maximum - 1:
                    print(f"{self.items[front_index]} <-", end
=" ")
                    front_index += 1

                front_index = 0

                while front_index <= rear_index:
                    print(f"{self.items[front_index]} <-", end
=" ")
                    front_index += 1

            print()
```

8. Advantages of Circular Queues

- Reuses memory that becomes available after deletions.
- Eliminates wasted space common in linear queues.

- No shifting of elements is required.
- Efficient for fixed-size buffer systems.
- Commonly used in operating systems, networking, and device buffering.

9. Disadvantages of Circular Queues

- More complicated than linear queues.
- Overflow and underflow conditions are harder to detect.
- Traversal logic is more complex due to wrap-around behavior.
- Fixed-size implementations still have a maximum capacity.

Summary

A **Circular Queue** is an improved version of a linear queue that connects the last position of the queue back to the first position.

Instead of wasting memory locations freed by dequeue operations, a circular queue reuses those positions by allowing the REAR and FRONT pointers to wrap around the array. This makes circular queues far more memory-efficient than traditional linear queues.

The key ideas are:

- REAR can return to the beginning of the array.
- FRONT can also wrap around after reaching the last index.
- Overflow occurs when REAR catches up to FRONT.
- Underflow occurs when the queue becomes empty.
- Traversal must account for wrap-around behavior.

Because of these properties, circular queues are widely used in real-world systems that require efficient memory utilization and continuous data processing.

Next, we will study the **Double-Ended Queue (Deque)**, a special queue that allows insertion and deletion from both ends. Unlike ordinary queues that strictly separate insertion and deletion points, a deque provides additional flexibility while still maintaining an ordered collection of elements.

References

https://drive.google.com/file/d/1A7WberDJxGmGtXpGZ7v-FFATGRIgKoQZ/view?usp=drive_link

https://www.programiz.com/dsa/circular-queue

https://www.geeksforgeeks.org/dsa/introduction-to-circular-queue/

Next Section:

10.3. Double-Ended Queue